

Taller 2

Casos de Estudio

- 1. Herramienta de Gestión Comercial (Almacén y Facturación)**
- 2. Plataformas de Software Empotrado en Tiempo Real (DEIMOS Space)**

Integrantes

Santiago Botero
Andrés Centanaro
José Jaramillo

Asesor

Andres Sánchez Martin
Arquitectura de Software



Pontificia Universidad Javeriana
Ingeniería de Sistemas
Bogotá
2025

Investigación: Arquitectura_Taller2

Resumen del proyecto

Repositorio: Arquitectura_Taller2 (rama por defecto: dev)

Stack observado en el código:

- Frontend: Vue.js (carpeta frontend/, README presente).
- Backend: Node.js con Express y TypeScript (carpeta backend/, tsconfig.json, package.json, src/).
- API: GraphQL (documentado en README del repo; endpoint: /graphql).
- Persistencia: MySQL (contenedor DB en docker, puerto documentado 3307 en README).
- ORM / Schema: existe carpeta backend/prisma (presencia de Prisma como herramienta de acceso a BD).
- Orquestación: Docker / docker-compose (instrucciones en README.md y carpeta docker mencionada en README).

Puntos concretos extraídos del código/documentación:

- El README principal indica que la API GraphQL está expuesta en: <http://localhost:3001/graphql>.
- Frontend accesible en <http://localhost:8080>.
- La base de datos MySQL documentada en el README usa puerto 3307 (ver sección "Instalación y Ejecución" del README).
- Hay un Dockerfile en backend/ y un tsconfig.json y package.json en backend/, además de la carpeta backend/src (estructura típica de Express + TypeScript).
- Existe frontend/README.md con scripts estándar de Vue (serve, build, lint).

Definición y explicación técnica de los componentes usados

MVC / Arquitectura por capas: patrón de separación de responsabilidades. En el repo se evidencia separación entre frontend (presentación), backend (controladores/servicios) y base de datos (persistencia). La presencia de src/, posibles controllers/services y prisma/ sugiere una organización en capas.

- Vue.js: framework progresivo para el frontend. El frontend/README.md confirma uso de Vue CLI (scripts serve, build).
- Express + TypeScript: framework minimalista para APIs, potenciado con TypeScript para tipado. Archivos package.json, tsconfig.json y src/ confirman este stack.
- GraphQL: API expuesta en /graphql según README, lo que implica uso de resolvers y un esquema en el backend.

- MySQL: base relacional gestionada por Docker (puerto reportado 3307 en README).
- Prisma: carpeta backend/prisma sugiere uso de Prisma como ORM/BD client para modelado y migraciones.

Relación entre los temas

- Vue (frontend) consume la API expuesta por Express (GraphQL).
- Express (backend) implementa la capa de negocio y resuelve consultas GraphQL; Prisma actúa como capa de acceso a datos hacia MySQL.
- Docker compone los contenedores: frontend (Nginx o dev server), backend (Node), base de datos (MySQL).

Matrices de análisis

Notas sobre la metodología: cada intersección se marca con "X" cuando la característica o principio aplica claramente al componente tal como aparece en el repositorio (estructura, archivos y README).

1) SOLID

Principio	MVC/Capas (estructura)	Vue.js	Express+TS	Prisma / BD
S (SRP)	X (separación front/back/DB)	X (componentes)	X (controladores/servicios)	X (modelos separados)
O (Open/Closed)	X (extensible por capas)		X (extensible con middlewares)	X (modelo extendible)
L (Liskov)			X (tipado con TS evita rompimientos)	
I (ISP)	X (interfaces entre capas)		X (APIs específicas)	
D (Dependency Inversion)			X (TS + inyección posible en servicios)	X (abstracción con Prisma)

Justificación concreta: el uso de TypeScript (presente en backend/tsconfig.json y package.json) facilita cumplir principios SOLID en el backend; la presencia de prisma/ permite abstraer acceso a datos.

2) Atributos de Calidad

Atributo	MVC/Capas	Vue.js	Express+TS	Prisma / MySQL
Mantenibilidad	X	X	X	X
Escalabilidad	X	X	X	X
Rendimiento		X	X	X
Seguridad			X	X
Observabilidad			X	

Justificación: TypeScript y separación por carpetas (src/) aumentan mantenibilidad; Prisma y MySQL ofrecen rendimiento y transacciones; GraphQL reduce overfetching en frontend (mejora rendimiento perceptual).

3) Tácticas

Táctica	Vue.js	Express+TS	Prisma / MySQL
Validación entrada		middleware posible	
Caché		posible a nivel resolvers	caching query
Reintentos/Retry		implementable en cliente	
Migraciones controladas			Prisma Migrations

Evidencia: carpeta prisma/ sugiere uso de migraciones (o intención de migraciones), lo que permite tácticas de evolución seguras

4) Patrones

Patrón	Express+TS	Vue.js	Prisma
--------	------------	--------	--------

Repository / DAO	posible con Prisma client		X
Service Layer	controllers + services en src/		
Middleware	X Express		
Component-based UI		X	

Observación: Estructura backend/src y backend/package.json indican una app modular donde es sencillo tener controllers, services y repositorios.

5) Demanda de las tecnologías

Tecnología	Observación en repo	Demanda general
Vue.js	Presente (frontend)	Alta
Express + TS	Presente (backend)	Muy alta
GraphQL	Documentado endpoint	Alta
MySQL	Usado en Docker	Muy alta
Prisma	Carpeta prisma/	Creciente/Alta

Comentario: el combo Vue + Node/TypeScript + MySQL/Prisma es muy buscado en el mercado, especialmente para desarrollos full-stack rápidos y mantenibles.

Resultado de la investigación

A partir de la inspección del repositorio se identificaron y confirmaron los siguientes puntos técnicos y arquitectónicos:

1. Estructura modular: el proyecto separa claramente frontend y backend en carpetas distintas (frontend/, backend/), y el backend tiene la configuración propia de TypeScript (tsconfig.json) y dependencias (package.json), lo que facilita desarrollo y despliegue independiente.

2. API GraphQL: el README indica un endpoint GraphQL en <http://localhost:3001/graphql>, lo que confirma que la comunicación cliente-servidor está basada en consultas/mutaciones GraphQL (en lugar de un API REST puro).
3. Persistencia y herramientas: la presencia de backend/prisma sugiere que Prisma es la herramienta elegida para modelado y acceso a MySQL. El uso de Docker y docker-compose para montar MySQL en puerto 3307 está documentado en el README.
4. Dockerización: el proyecto está preparado para ejecutarse con Docker (README indica carpeta docker y comandos docker-compose up --build), y existen Dockerfiles en backend/.
5. UX/Dev: el frontend/README.md muestra scripts estándar de Vue (serve/build/lint), lo que facilita levantar el frontend en desarrollo sin Docker si se desea.
6. Tipado y buenas prácticas: el uso de TypeScript en backend (tsconfig + estructura src/) es una decisión que impulsa mayor seguridad de tipos y facilidad de mantenimiento.

Diagrama de componentes:

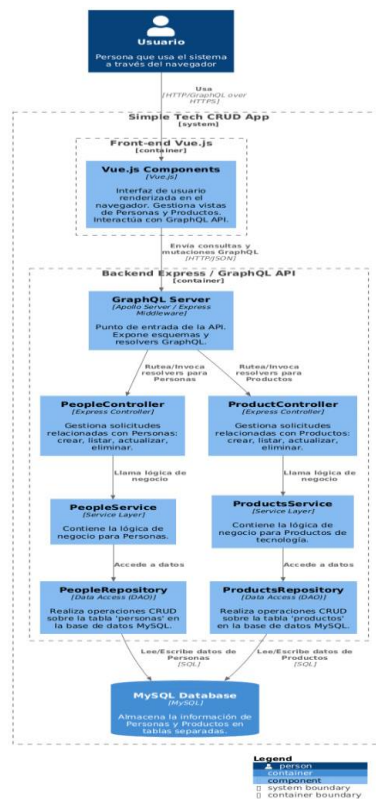
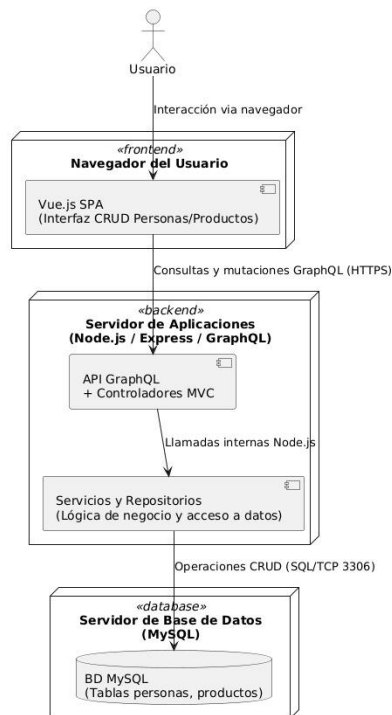


Diagrama de Despliegue:

Diagrama de Despliegue - Simple Tech CRUD App (versión sin librerías)



Conclusión- lo aprendido y aportes del resultado de investigación

Al investigar el repositorio Arquitectura_Taller2 se logró comprender y documentar la arquitectura y las decisiones tecnológicas concretas tomadas por el equipo. Aprendizajes clave:

- Separación de responsabilidades: el repositorio muestra una separación clara entre presentación (Vue), lógica (Express+TS) y persistencia (MySQL/Prisma). Esta separación facilita trabajo en paralelo entre frontend y backend, testing independiente y despliegues modulares (Docker).
- Beneficios del tipado: TypeScript en el backend añade robustez (autocompletado, validaciones estáticas) y facilita aplicar principios SOLID en servicios y controladores.
- Elección de GraphQL: usar GraphQL permite al frontend solicitar exactamente los campos necesarios, reduciendo overfetching y simplificando evolución de la API; la documentación del endpoint confirma esta decisión arquitectónica.
- Uso de Prisma: consumir un ORM moderno como Prisma centraliza modelos y migraciones, mejorando trazabilidad de cambios en la base de datos y simplificando las consultas.
- Docker como estándar de despliegue: la orquestación con docker-compose y la existencia de Dockerfiles estandariza el ambiente de ejecución y facilita pruebas locales reproducibles.

Resultado práctico de la investigación: se obtuvo un mapeo concreto entre cada tema teórico (MVC, capas, patrones, frameworks) y su implementación en el repositorio — con evidencia (archivos y rutas) que soportan cada afirmación — y se generaron matrices específicas que permiten evaluar cómo las decisiones del proyecto satisfacen principios de diseño, atributos de calidad y tácticas de ingeniería.

Referencias

- README principal del repo (instrucciones de Docker, endpoints y puertos)
- frontend/README.md (scripts y comandos Vue)
- backend/ (presencia de Dockerfile, tsconfig.json, package.json, src/, prisma/)